

Tool Decorators: Keeping Code and Documentation in Sync

When building AI agents, one of the most challenging aspects is maintaining accurate documentation about the tools an agent can use. Every time we modify a tool's parameters or behavior, we need to update its description and parameter schema. If these get out of sync, our agent might try to use tools incorrectly, leading to failures that can be hard to debug. Let's explore how we can solve this problem elegantly using Python decorators.

The Challenge of Tool Documentation

Imagine we're writing a tool for our agent to use. Without any special handling, our might write something like this:

```
def greet(name: str) -> str:
    """Greet someone by name"""
    return f'Hello, {name}!'

def greet(name: str, verbose: bool = False) -> str:
    """Greet someone by name, with an optional verbose flag"""
    if verbose:
        return f'Hello, {name}! Verbose mode is on.'
    else:
        return f'Hello, {name}!'

# Example usage
print(greet('Alice'))
print(greet('Bob', verbose=True))
```

This approach has several problems. If we add a new parameter to `greet()`, we need to remember to update the parameter schema. If we change the function's behavior, we need to update the description. It's easy for these to become out-of-sync over time.

The Power of Decorators

Luckily, we can use Python's decorator system to automatically update and maintain this information. Here's how it works:

```
def register_tool(tool: Callable, description: str, schema: dict) -> Callable:
    """Registers a tool with its description and schema"""
    def wrapper(*args, **kwargs):
        # The function gets the title to read and returns the schema
        title = tool.__doc__.split('\n')[0].strip()
        schema['title'] = title
        return tool(*args, **kwargs)
    wrapper.__doc__ = description
    wrapper.__schema__ = schema
    return wrapper
```

Our decorator registers the function and automatically:

- 1. Updates the function's docstring to include the description.
- 2. Updates the function's parameter schema to include the schema.
- 3. Updates the function's parameter schema to include the schema.
- 4. Registers the tool in a central registry.

Implementing the Decorator

Let's walk through the implementation by first understanding the decorator and then registering the higher function that does the heavy lifting.

```
def register_tool(tool: Callable, description: str, schema: dict) -> Callable:
    """Registers a tool with its description and schema"""
    def wrapper(*args, **kwargs):
        # The function gets the title to read and returns the schema
        title = tool.__doc__.split('\n')[0].strip()
        schema['title'] = title
        return tool(*args, **kwargs)
    wrapper.__doc__ = description
    wrapper.__schema__ = schema
    return wrapper
```

The `register_tool()` decorator is designed to transform regular Python functions into callable tools for an agent system. Here's what's happening in detail:

- 1. The decorator accepts three arguments: a function `tool`, a string `description`, and a dictionary `schema`.
- 2. It defines an inner function `wrapper()` that takes the actual function being decorated (`tool`) as its parameter.
- 3. Within the `wrapper` function, it first calls `tool.__doc__.split('\n')[0].strip()` to extract the first line of the function's docstring. It then updates the `schema` dictionary with this title.
- 4. It then registers the tool in a global dictionary `registry`, using the tool's name as the key. The `registry` is a dictionary that maps tool names to their schemas.
- 5. The decorator then returns the `wrapper` function, which is now the new callable tool. The `wrapper` function is configured to look up the tool's schema in the `registry` and use it to validate the arguments passed to the tool.
- 6. Finally, it returns the original function `tool`, allowing it to be called normally with all its original arguments.

This pattern ensures that the information needed for the tool's validation and its registration is kept together, making the decorator much more maintainable.

Now, let's explore the higher function that does the heavy lifting.

```
def register_tool(tool: Callable, description: str, schema: dict) -> Callable:
    """Registers a tool with its description and schema"""
    def wrapper(*args, **kwargs):
        # The function gets the title to read and returns the schema
        title = tool.__doc__.split('\n')[0].strip()
        schema['title'] = title
        return tool(*args, **kwargs)
    wrapper.__doc__ = description
    wrapper.__schema__ = schema
    return wrapper
```

The `register_tool()` decorator performs registration on the decorated function. It automatically extracts the metadata needed to describe the tool to the agent.

- 1. The decorator extracts the title from the function's docstring and updates the schema.
- 2. It updates the schema with the function's description and its parameter schema.
- 3. It registers the tool in a global dictionary `registry`, using the tool's name as the key.
- 4. It returns the original function `tool`, allowing it to be called normally with all its original arguments.

This decorator registration enables tools to be defined with minimal boilerplate while providing rich metadata for the agent system to understand how to call each tool properly. The tool description will always match the function's signature and description, ensuring that the agent has the most accurate information available.

Why Create a Decorator?

We will use the decorator approach in many different agent-based tools. Here is why:

- 1. **Single Source of Truth:** The function itself becomes the authoritative source for all tool information. The decorator describes what the tool does, the type hints define its parameters, and the implementation shows how it works.
- 2. **Automatic Updates:** When we modify the function's signature or documentation, the tool registration automatically stays in sync. The new function through code to update parameter schema.
- 3. **Clear Organization:** The tool system allows us to organize tools and their documentation. We can easily get at "The parameter" tools or "The description" tools or "The schema" tools.
- 4. **Improved Development Experience:** The code we write in standard Python functions with standard documentation. The decorator handles all the complexity of making them available to our agent.

Consider how this simplifies adding a new parameter:

```
def greet(name: str) -> str:
    """Greet someone by name"""
    return f'Hello, {name}!'

def greet(name: str, verbose: bool = False) -> str:
    """Greet someone by name, with an optional verbose flag"""
    if verbose:
        return f'Hello, {name}! Verbose mode is on.'
    else:
        return f'Hello, {name}!'

# Example usage
print(greet('Alice'))
print(greet('Bob', verbose=True))
```

The tool's parameter schema is automatically updated to include the new `verbose` parameter on `greet()` because it has a default value. We didn't need to manually update any registration code or documentation.